

Einführung in die Programmierung (WIN+BIT)

Objekt-orientiertes Design, grundlegende Prinzipien

Prof. Dr. Johannes C. Schneider / Prof. Dr. Markus Eiglsperger



Aufgepasst?



pingo.coactum.de, Code **347459** eingeben

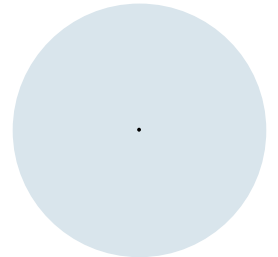
oder direkt <https://pingo.coactum.de/347459>

– Geht auch vom Handy aus



Disclaimer: Lesen Sie die [Datenschutzhinweise](#) selbst und entscheiden Sie, ob Sie bei den Umfragen mitmachen wollen.

OOD-Prinzip: Klassen und Instanzen



Objekt-orientiertes Design (OOD)

- Im Objekt-orientierten Design bildet man Zusammenhänge aus der "echten Welt" über Beziehungen zwischen Klassen und Objekten ab.

Klassen (classes)

- *Selbstdefinierter* (Referenz-)Datentyp
- Für *Modellierung* neuer Strukturen
- *Kernkonzept* der objektorientierten Programmierung
- *Sammlung* von Variablen ("Attributen") und Methoden
- *Konkrete Instanz* einer Klasse heißt **Objekt**
 - daher auch **Objektorientiertes Programmieren (OOP)**

Klasse versus Instanz

allgemein

Klasse

- Beschreibung einer allgemeinen Struktur z.B. durch die Attribute
- *"Eine Adresse besteht aus Name, Straße, Ort, PLZ."*

konkret

Instanz / Objekt

- Eine mögliche, konkrete Ausprägung einer bestimmten Klasse (Struktur)
- Es gibt (meist) mehrere Möglichkeiten, eine Klasse zu instanziiieren
- *"Johannes Schneider, Alfred-Wachtel-Str. 8, 78462, Konstanz"* ist eine mögliche Ausprägung der links definierten Adresse.
- *"Beate Beispiel, Lindenstraße 1, 67346, Speyer"* eine andere.

Weitere Beispiele

allgemein

Klasse

- *"Eine Zeichenkette (String) besteht aus mehreren Zeichen und hat eine Funktion, welche die Länge zurückgibt."*
- *"Ein Auto hat einen Hersteller, eine Typenbezeichnung, ein Baujahr und eine Farbe."*

"Instanz zu Klasse" definiert eine **"ist ein"**-Beziehung:

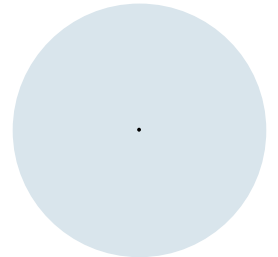
- Ein Porsche 911 **ist ein** Auto.
- "Hallo" **ist eine** Zeichenkette
- *"Johannes Schneider, Alfred-Wachtel-Str. 8, 78462, Konstanz"* **ist eine** Adresse

konkret

Instanzen / Objekte

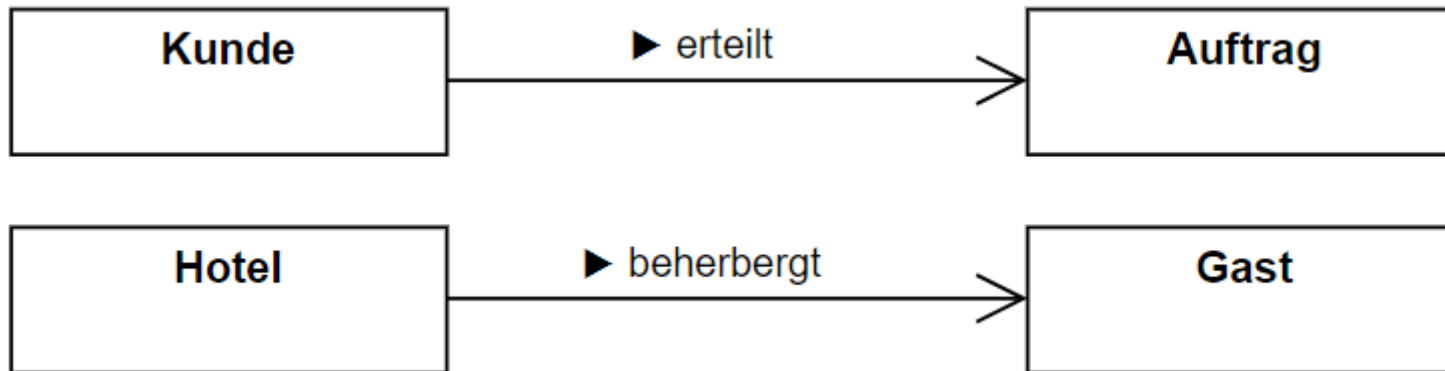
- "Hallo" mit `"Hallo".length() == 5`
- "" mit `"".length() == 0`
- "Citroën, 2CV, 1971, grün"
- "Porsche, 911, 1965, blau"

OOD-Prinzip: Assoziation



Assoziation

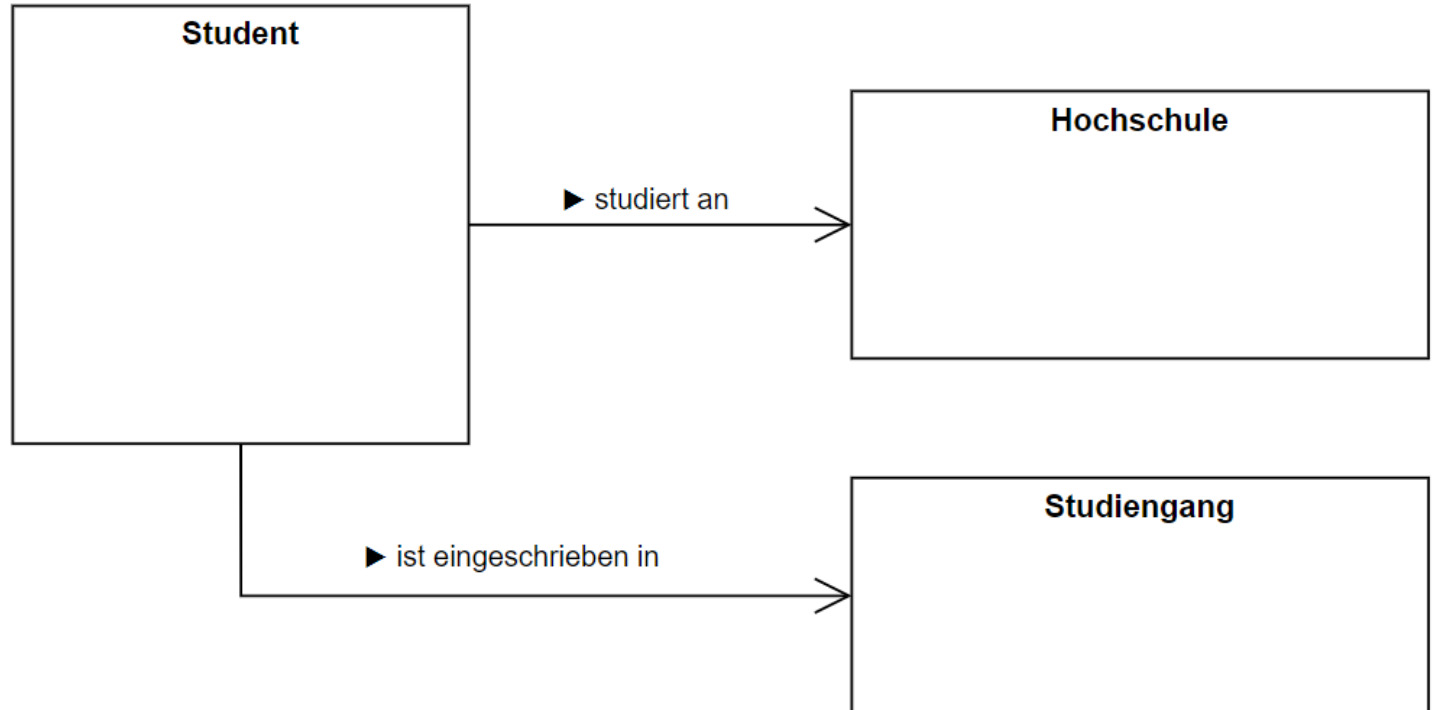
- Eine Assoziation drückt eine *Beziehung* zwischen zwei *Klassen* aus.
- Beispiele:
 - Ein *Kunde** *erteilt* einen *Auftrag*
 - Ein *Hotel* *beherbergt* (einen oder mehrere) *Gäste**
- In UML-Schreibweise als Linie bzw. Pfeil dargestellt



*auch gemeint: Kund:in, Gäst:innen

Beispiel Student, Studiengang, Hochschule

- Klassen: **Substantive**,
Beziehung (meistens) **Verben**
- Ein **Student*** *studiert* an einer
Hochschule
- Ein **Student*** *ist eingeschrieben*
in einen **Studiengang**



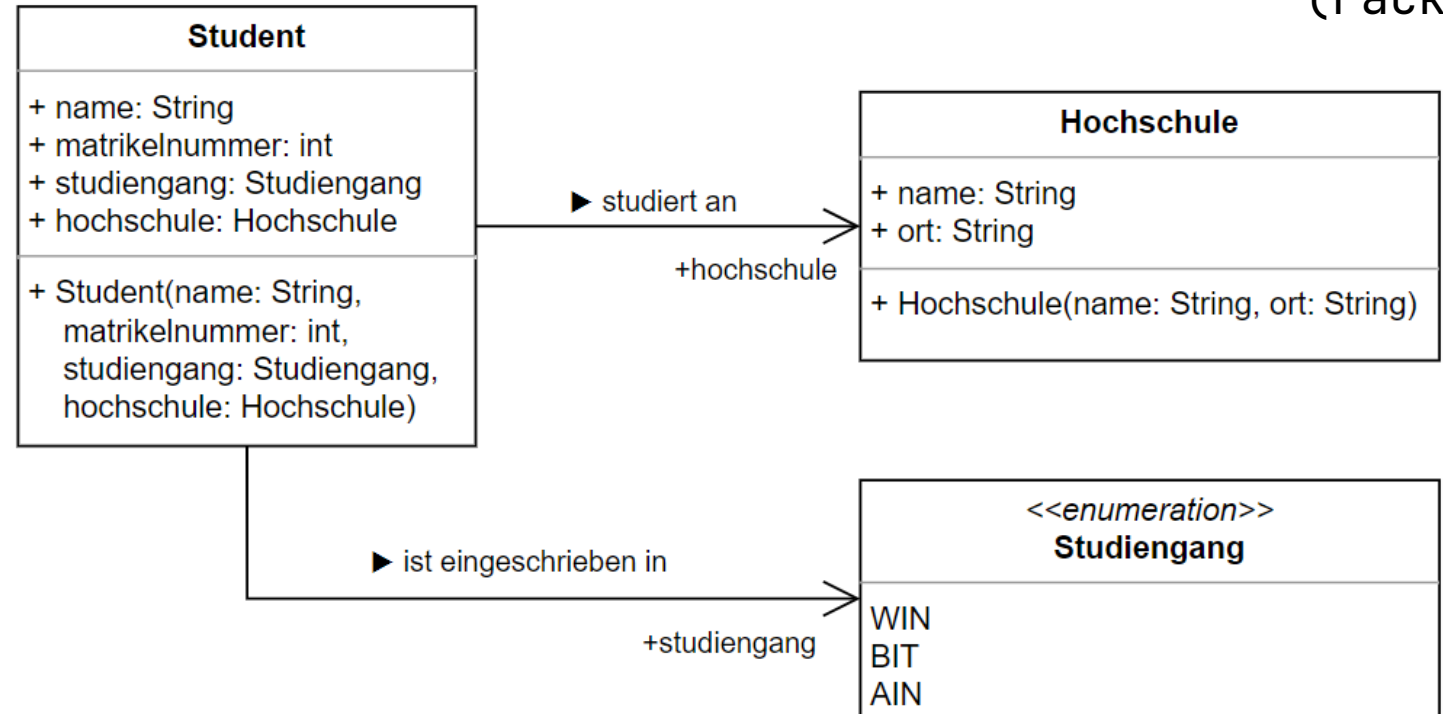
*auch gemeint: Student:innen, benutzen Student als gültigen Java-Klassennamen

Beispiel Student, Studiengang, Hochschule

Modellierung in Java:

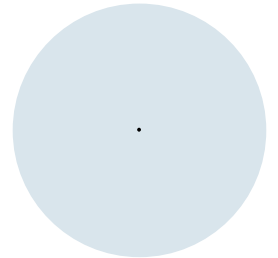
- Student, Hochschule und Studiengang* als Klassen
- Beziehung über Attribut des jeweiligen Typs:
 - Student hat Attribut studiengang vom Typ Studiengang
 - Student hat Attribut hochschule vom Typ Hochschule

* Studiengang ist spezielle Klasse, nämlich Enum, da nur fest definierte Liste von Studiengängen zugelassen



So modelliert man das in UML. Das müssen Sie für die Prüfung nicht können. Es hilft aber gut, die Zusammenhänge zu visualisieren.

OOD-Prinzip: Vererbung

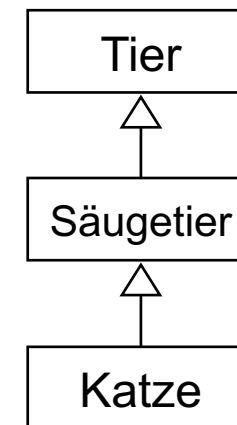
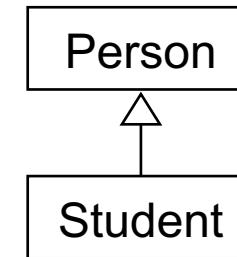


Vererbung

- **Klassen können von anderen Klassen erben** und so Implementierung (Attribute und Methoden) übernehmen
- **Spezialisierung:** Klasse A, die von Klasse B erbt, ist spezieller als Klasse B, da sie mindestens alles übernimmt was B schon hat, aber ggf. noch mehr anbietet.

Vererbungsbeziehung

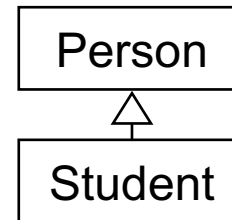
- **Vererbung (Spezialisierung) definiert eine "ist-ein(e)"-Beziehung**
 - Klasse Student erbt von Klasse Person bedeutet:
(Jede:r) Student:in **ist eine** Person
 - Im Folgenden stellvertretend Maskulinum "Student" verwendet
 - Umgekehrt wäre es falsch. Klasse Person erbt von Klasse Student würde bedeuten: Jede Person **ist ein** Student.
- Mehrfachvererbung:
 - Klasse Katze erbt von Klasse Säugetier erbt von Klasse Tier:
 - (Jede) Katze **ist ein** Säugetier.
 - (Jedes) Säugetier **ist ein** Tier.
 - auch: (Jede) Katze **ist ein** Tier.
 - Vererbung ist eine transitive Relation: Wenn "A erbt von B" und "B erbt von C", dann auch "A erbt von C"



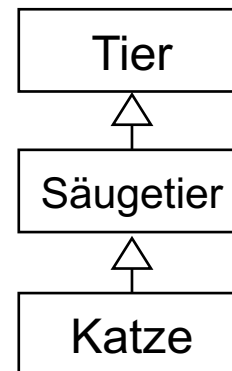
Oberklasse und Unterklasse

- englisch: *superclass* und *subclass*
- Diese Begriffe sind immer *relativ* zu sehen:

- Person ist Oberklasse von Student
- Student ist Unterklasse von Person



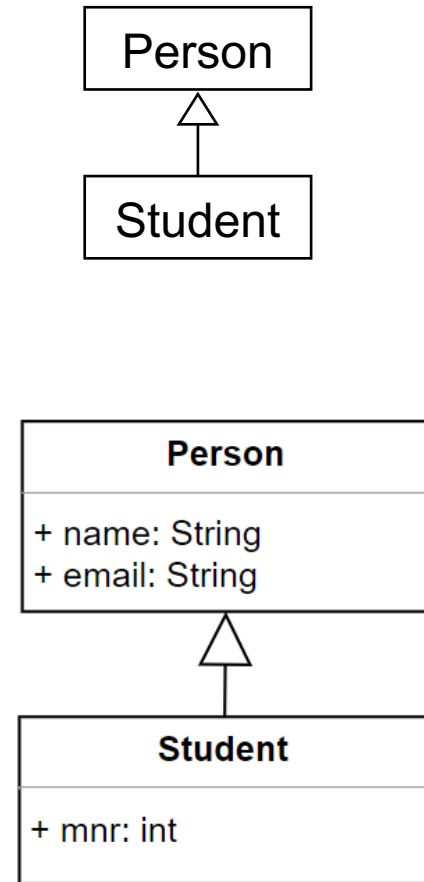
- Säugetier ist Unterklasse von Tier
- Säugetier ist Oberklasse von Katze



Beispiel Person und Student, Attribute

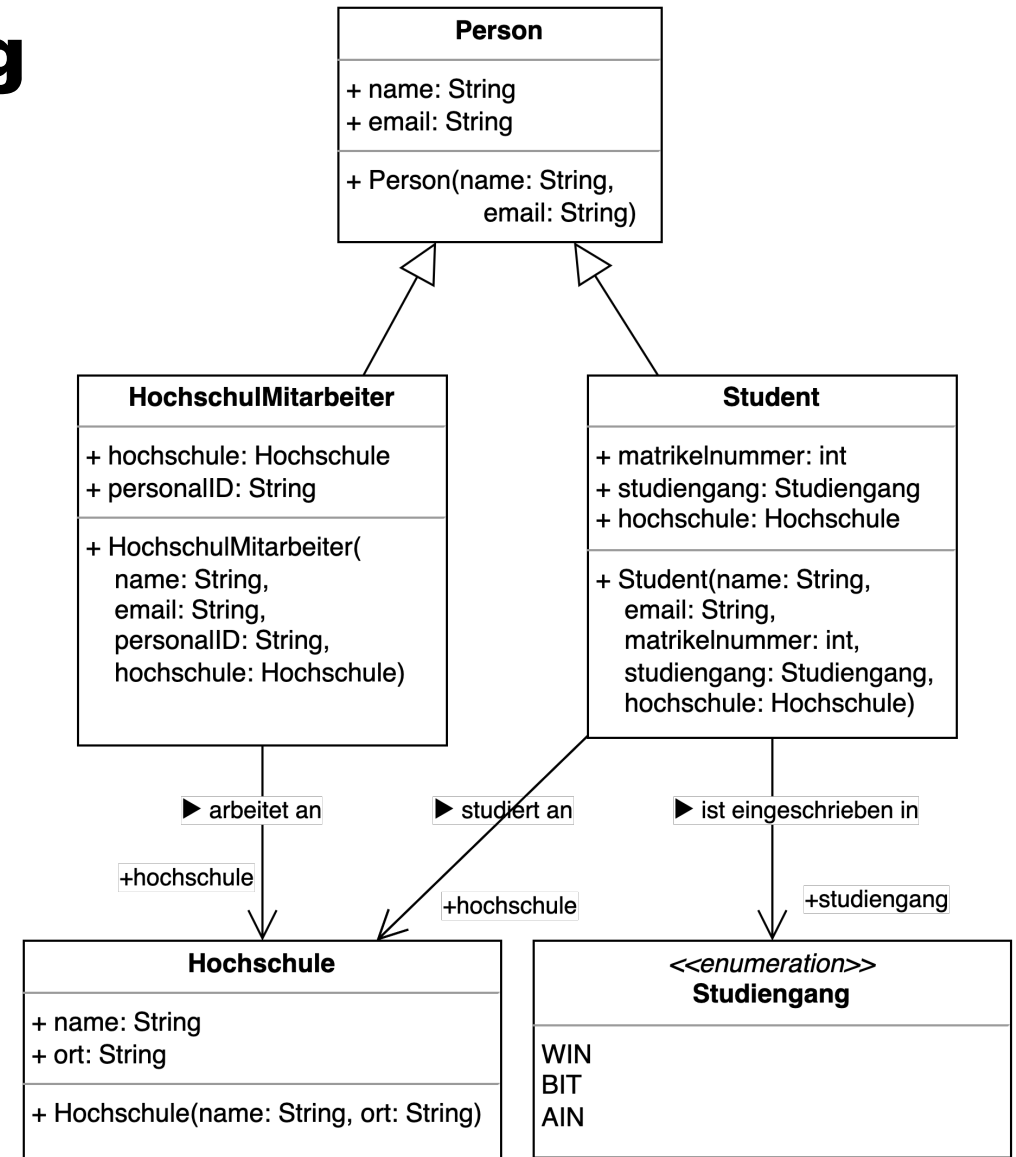
- Jeder Student ist eine Person
- Ein Student hat also alle Eigenschaften, die auch eine Person hat
- Eigenschaften von Student und Person:
 - Name
 - E-Mail-Adresse
- Eigenschaften, die nur ein Student hat
 - Matrikelnummer
- Eigenschaften modelliert man in Java als Attribute

mit Attributen

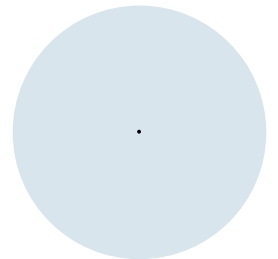


Objektorientierte Modellierung

- Beispiel für objektorientierte Modellierung:
Wir bilden Zusammenhänge aus der "echten Welt" über Beziehungen zwischen Klassen und Objekten ab.
 - Assoziation: Klassen stehen in einem Verhältnis zueinander (z.B. "studiert an")
 - Vererbung: Klasse A ist eine Spezialisierung von Klasse B (z.B. ist jede:r Student:in auch eine Person)
- weitere Beziehungsarten zwischen Klassen:
 - Aggregation (Klasse A besitzt Klasse B)
 - Komposition (Klasse A setzt sich aus Klassen B, C, D, ... zusammen)



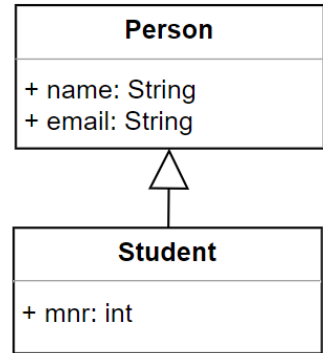
Vererbung (Umsetzung in Java)



Vererbung

- **Klassen können von anderen Klassen erben** und so Implementierung (Attribute und Methoden) übernehmen
- **Spezialisierung:** Klasse A, die von Klasse B erbt, ist spezieller als Klasse B, da sie mindestens alles übernimmt was B schon hat, aber ggf. noch mehr anbietet.
- In Java:
 - Schlüsselwort extends: `class A extends B { ... }`
 - Eine Klasse kann nur von höchstens einer anderen Klasse erben (keine Mehrfachvererbung)

Beispiel Person und Student, Attribute



- Für Vererbung verwendet man in Java das Schlüsselwort `extends`

```
public class Person {
    public String name;
    public String email;
}
```

Person-Instanzen haben nur die Attribute `name` und `email`

```
Person max = new Person();
max.name = "Max";
max.email = "max@example.org";

max.mnr = 123456;
```

Compilerfehler: Cannot resolve symbol `mnr`

```
public class Student extends Person {
    public int mnr;
}
```

Student-Instanzen erben die Attribute `name` und `email` und haben ein zusätzliches Attribut `mnr`

```
Student beate = new Student();
beate.name = "Beate";
beate.email = "beate@example.org";
beate.mnr = 123456;
```



code17.
student1
(Package)

Beispiel Person und Student, Methoden

- Auch Methoden werden vererbt
- Wir wollen Methoden hinzufügen, die E-Mail-Adresse und Matrikelnummer validieren
 - geben true bzw. false zurück, je nachdem, ob die E-Mail-Adresse bzw. Matrikelnummer gültig ist
- E-Mail-Adresse (sehr einfacher Check, kann man noch erweitern, mehr dazu in SWEN1)

```
public boolean hasValidEmail() {  
    return this.email != null && this.email.contains("@");  
}
```

- Matrikelnummer

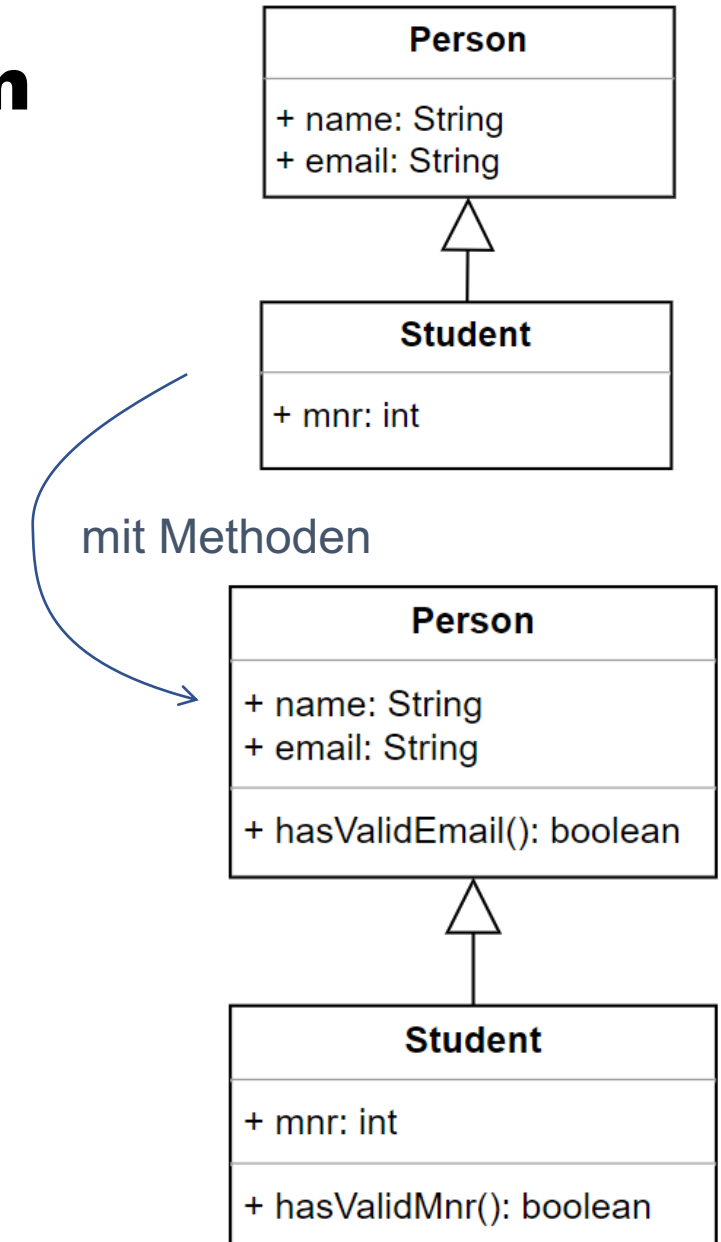
```
public boolean hasValidMnr() {  
    return this.mnr > 0;  
}
```

Beispiel Person und Student, Methoden

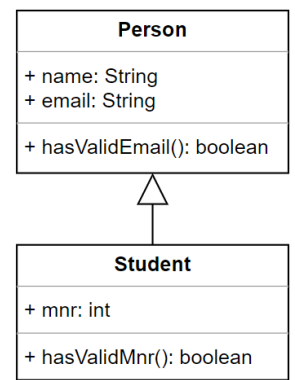
- In welche Klasse kommt welche Methode?

```
public boolean isValidEmail() {  
    return this.email != null  
        && this.email.contains("@");  
}
```

```
public boolean isValidMnr() {  
    return this.mnr > 0;  
}
```



Beispiel Person und Student, Methoden



```
public class Person {
    public String name;
    public String email;
    public boolean isValidEmail() {
        return this.email != null
            && this.email.contains("@");
    }
}
```

Person-Instanzen haben nur die Methode `isValidEmail()`

```
Person max = new Person();
max.isValidEmail();
max.isValidMnr();
```

```
public class Student extends Person {
    public int mnr;
    public boolean isValidMnr() {
        return this.mnr > 0;
    }
}
```

Student-Instanzen erben `isValidEmail()` und haben zusätzliche Methode `isValidMnr()`

```
Student beate = new Student();
beate.isValidEmail();
beate.isValidMnr();
```

Compilerfehler: Cannot resolve method `isValidMnr`

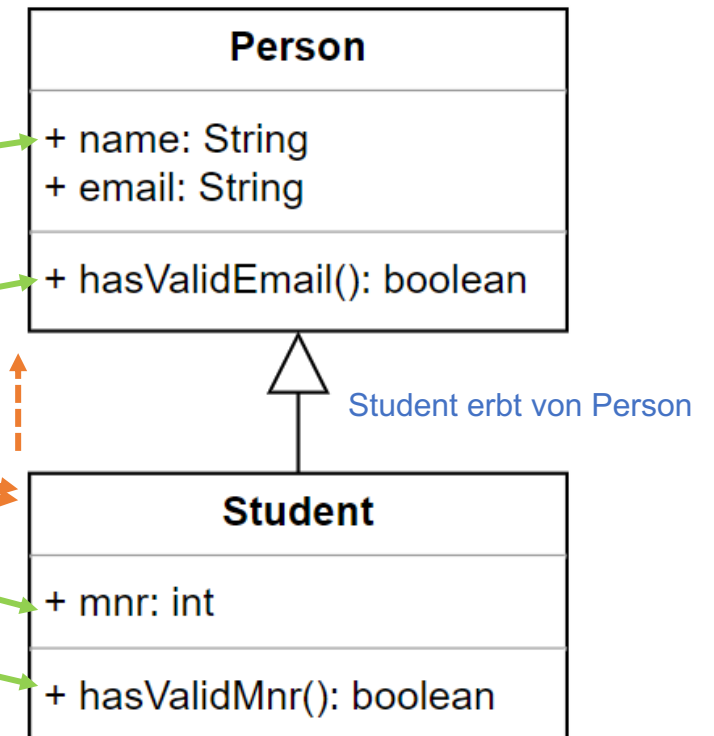


code17.
student2
(Package)

Zugriff auf geerbte Attribute und Methoden

- Java sieht zuerst bei Klasse selbst nach, ob es die Methode dort gibt, ansonsten bei Oberklasse(n)

```
Student student = new Student();  
student.name = "Alex";  
System.out.println(student.isValidEmail());  
System.out.println(student.mnr);  
if (!student.isValidMnr()) {  
    System.out.println("Matrikelnummer ungültig");  
}
```



Parameterloser Konstruktor bei Vererbung (1)

- Wenn die Oberklasse einen parameterlosen Konstruktor anbietet, muss man nichts weiter beachten.

```
public class Person {  
    public String name;  
    public String email;  
  
    public Person() {  
  
    }  
}
```

```
public class Student extends Person {  
    public int mnr;  
  
    public Student() {  
  
    }  
}
```

Funktioniert:

```
Student alex = new Student();
```

Wichtig: Der Student()-Konstruktor ruft automatisch den Person()-Konstruktor auf, siehe Beispiel



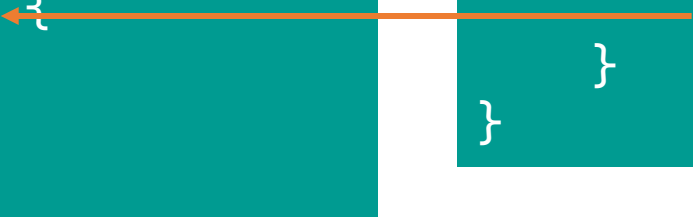
code17.
student3
(Package)

Parameterloser Konstruktor bei Vererbung (2)

- Wenn die Oberklasse einen parameterlosen Konstruktor anbietet, muss man nichts weiter beachten.

```
public class Person {  
    public String name;  
    public String email;  
  
    public Person() {  
  
    }  
}
```

```
public class Student extends Person {  
    public int mnr;  
  
    public Student() {  
        super();  
    }  
}
```



Funktioniert:

```
Student alex = new Student();
```

Wichtig: Der Student()-Konstruktor ruft automatisch den Person()-Konstruktor auf.

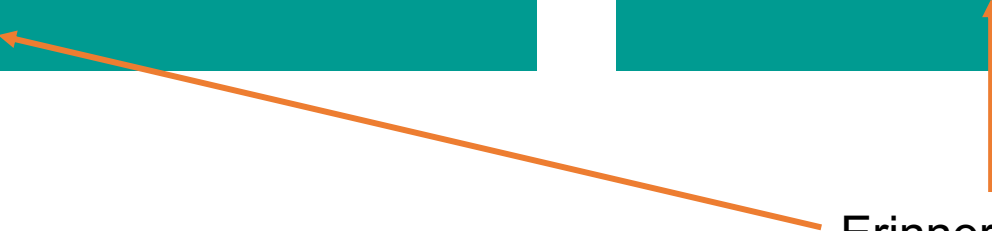
Das kann man auch explizit so hinschreiben (es passiert genau das gleiche wie bei voriger Folie), muss man aber nicht.

Parameterloser Konstruktor bei Vererbung (3)

- Wenn die Oberklasse einen parameterlosen Konstruktor anbietet, muss man nichts weiter beachten.

```
public class Person {  
    public String name;  
    public String email;  
}
```

```
public class Student extends Person {  
    public int mnr;  
}
```



Erinnerung: Man kann den parameterlosen Konstruktor auch weglassen, er wird dann von Java automatisch erzeugt. Es passiert dann dasselbe wie bei den vorherigen Beispielen.

Funktioniert immer noch:

```
Student alex = new Student();
```

Konstruktor mit Parametern bei Vererbung

- Wenn die Oberklasse **keinen** parameterlosen Konstruktor anbietet, muss man mittels super einen der angebotenen Konstruktoren der Oberklasse aufrufen, ansonsten gibt es einen Fehler:

```
public class Person {  
    public String name;  
    public String email;  
  
    public Person(String name,  
                  String email) {  
        this.name = name;  
        this.email = email;  
    }  
}
```

```
public class Student extends Person {  
    public int mnr;  
  
    public Student() {  
    }  
}
```

Compilerfehler: There is no default constructor available in Person

Konstruktor mit Parametern bei Vererbung

- Wenn die Oberklasse **keinen** parameterlosen Konstruktor anbietet, muss man mittels `super` einen der angebotenen Konstruktoren der Oberklasse aufrufen.

```
public class Person {  
    public String name;  
    public String email;  
  
    public Person(String name,  
                  String email) {  
        this.name = name;  
        this.email = email;  
    }  
}
```

```
public class Student extends Person {  
    public int mnr;  
  
    public Student() {  
        super(null, null);  
    }  
}
```

So funktioniert es zwar, aber es geht besser...

Konstruktor mit Parametern bei Vererbung

- Wenn die Oberklasse **keinen** parameterlosen Konstruktor anbietet, muss man mittels `super` einen der angebotenen Konstruktoren der Oberklasse aufrufen.

```
public class Person {  
    public String name;  
    public String email;  
  
    public Person(String name,  
                   String email) {  
        this.name = name;  
        this.email = email;  
    }  
}
```

```
public class Student extends Person {  
    public int mnr;  
  
    public Student(String name,  
                   String email, int mnr) {  
        super(name, email);  
        this.mnr = mnr;  
    }  
}
```

Komplettes, typisches Beispiel, wie man in Datenklassen den Konstruktor-Aufruf an die Oberklasse weiterleitet, um alle Attribute direkt beim Erzeugen der Klasse zu setzen.



code17.
student4
(Package)

Konstruktor-Aufruf bei Vererbung, Besonderheiten

- Der `super(...)`-Aufruf muss im Konstruktor der Unterklasse (=Klasse, die von einer anderen Klasse erbt) als allererster "Befehl" stehen
 - ähnlich wie `this(...)` zum Aufruf eines anderen Konstruktoren derselben Klasse
- Bietet die Oberklasse mehrere Konstruktoren an, kann man sich für den `super(...)`-Aufruf einen davon auswählen.

```
public class Student extends Person {  
    public int mnr;  
  
    public Student(String name,  
                    String email, int mnr) {  
        this.mnr = mnr;  
        super(name, email);  
    }  
}
```

Compilerfehler: Call to super must be first statement in constructor body